

Assembly Programming

Readings

- *Introduction to Computing Systems: From Bits and Gates to C and Beyond* by Patt & Patel
 - Chapter 5, 6
 - Appendix A – LC3 ISA

Addressing Example in LC-3

- What is the final value of R3?

Address	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
x30F6	1	1	1	0	0	0	1	1	1	1	1	1	1	1	1	0	R1 = PC - 3 = 0x30F7 - 3 = 0x30F4
x30F7	0	0	0	1	0	1	0	0	0	1	1	0	1	1	1	1	R2 = R1 + 14 = 0x30F4 + 14 = 0x3102
x30F8	0	1	1	1	0	1	0	1	1	1	1	1	1	0	1	1	M[PC - 5] = M[0x30F4] = 0x3102
x30F9	1	0	1	1	0	1	0	0	1	0	1	0	0	0	0	0	R2 = 0
x30FA	0	0	1	1	0	1	0	0	1	0	1	1	0	1	0	1	R2 = R2 + 5 = 5
x30FB	1	1	1	1	0	1	0	0	0	1	1	1	1	1	1	1	M[R1 + 14] = M[0x30F4 + 14] = M[0x3102] = 5
x30FC	0	1	0	1	0	1	1	1	1	1	1	1	1	1	1	1	R3 = M[M[PC - 9]] = M[M[0x30FD - 9]] =
																	M[M[0x30F4]] = M[0x3102] = 5

- The final value of R3 is 5

Control Flow Instructions

- In LC-3 and in MIPS there are three types of opcodes
 - Operate
 - Data movement
 - Control

Control Flow Instructions

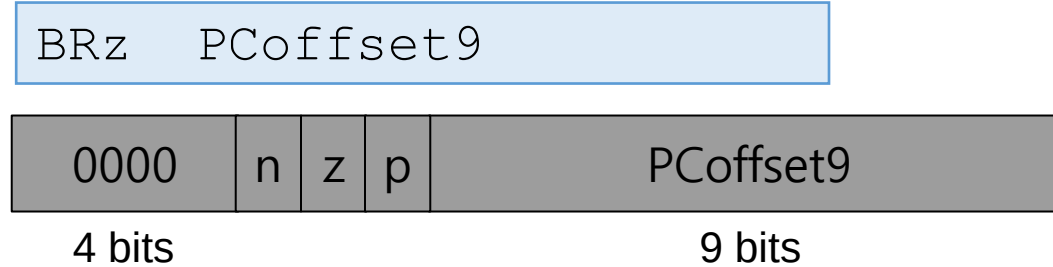
- Allow a program to execute **out of sequence**
- Conditional branches and jumps
 - **Conditional branches** are used to **make decisions**
 - E.g., if-else statement
 - In LC-3, three **condition codes** are used
 - **Jumps** are used to implement
 - **Loops**
 - **Function calls**
 - **JMP** in LC-3 and **j** in MIPS

Condition Codes in LC-3

- Each time one GPR (R0-R7) is written, **three single-bit registers** are updated
- Each of these **condition codes** are either set (set to 1) or cleared (set to 0)
 - If the written value is **negative**
 - **N** is set, Z and P are cleared
 - If the written value is **zero**
 - **Z** is set, N and P are cleared
 - If the written value is **positive**
 - **P** is set, N and Z are cleared
- SPARC and x86 are examples of ISAs that use condition codes

Conditional Branches in LC-3

- BRz (Branch if Zero)



- n, z, p = **which condition code is tested** (N, Z, and/or P)
 - n, z, p : instruction bits to identify the condition codes to be tested
 - N, Z, P: values of the corresponding condition codes
- PCoffset9 = immediate or constant value
- **if $((n \text{ AND } N) \text{ OR } (p \text{ AND } P) \text{ OR } (z \text{ AND } Z))$**
 - **then $PC \leftarrow PC^\dagger + \text{sign-extend}(\text{PCoffset9})$**
- Variations: BRn, BRz, BRp, BRzp, BRnp, BRnz, BRnzp

[†] This is the incremented PC

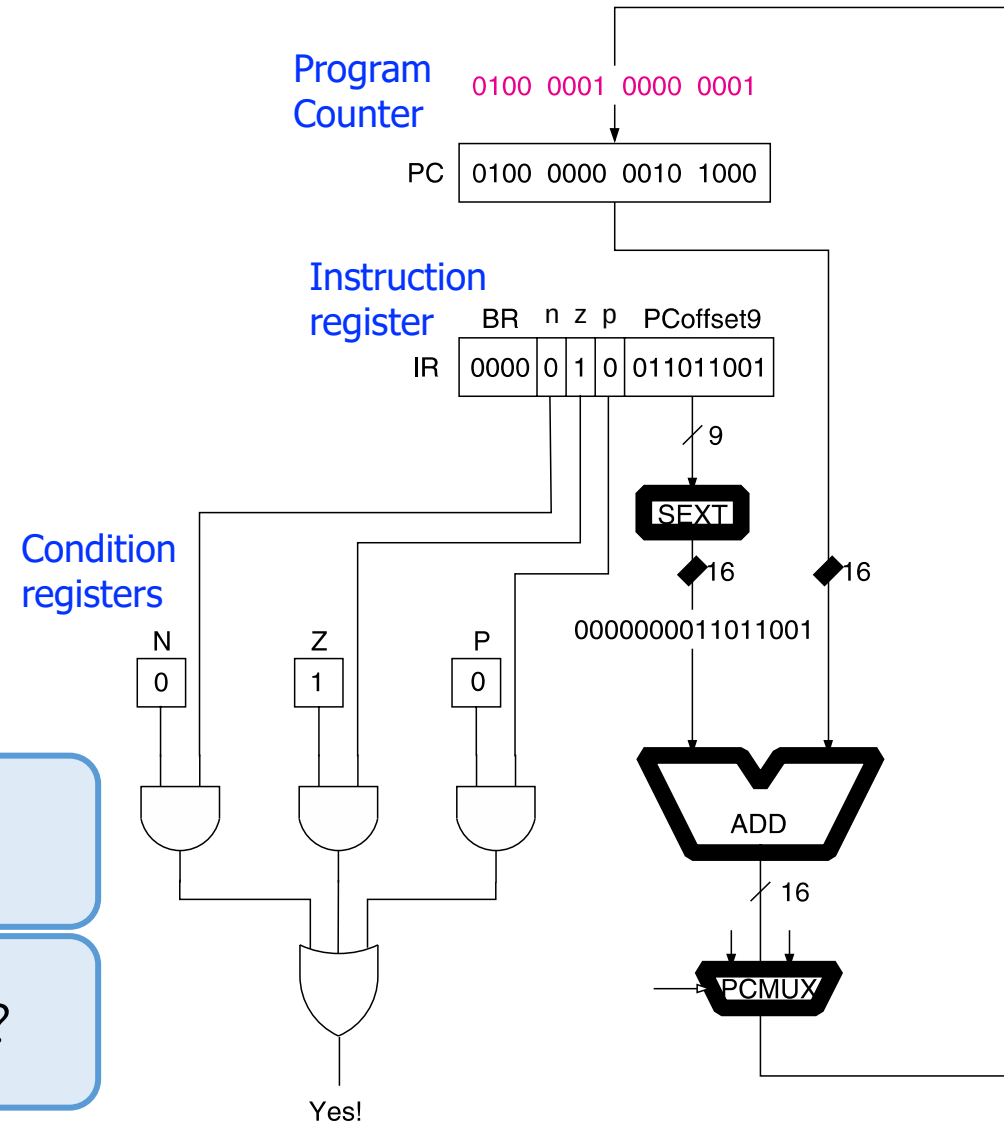
Conditional Branches in LC-3

- BRz

BRz 0x0D9

What if $n = z = p = 1$?*
(i.e., BRnzp)

And what if $n = z = p = 0$?



*n, z, p are the instruction bits to identify the condition codes to be tested

Branch If Equal in MIPS and LC-3

MIPS assembly

```
beq  $s0, $s1, offset
```

LC-3 assembly

```
NOT  R2, R1
```

```
ADD  R3, R2, #1
```

```
ADD  R4, R3, R0
```

```
BRz  offset
```

**Subtract
(R0 - R1)**

- This is an example of **tradeoff** in the instruction set
 - The same functionality requires **more instructions in LC-3**
 - But, the **control logic** requires **more complexity in MIPS**

JSR

Jump to Subroutine

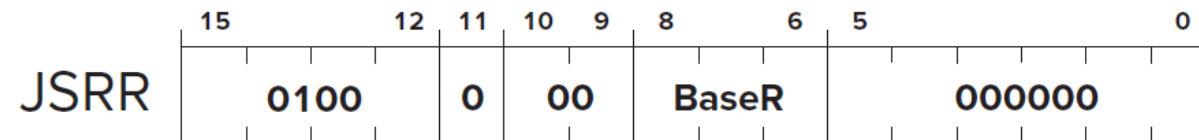
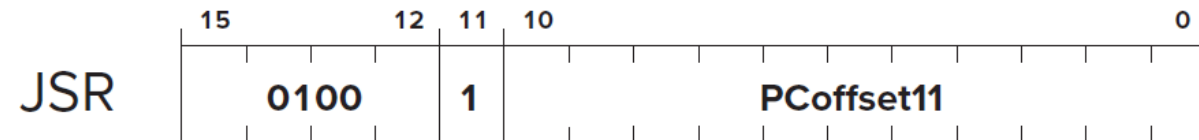
JSRR

Assembler Formats

JSR LABEL

JSRR BaseR

Encoding



Operation

```
TEMP = PC;†  
if (bit[11] == 0)  
    PC = BaseR;  
else  
    PC = PC† + SEXT(PCoffset11);  
R7 = TEMP;
```

JMP

Jump

RET

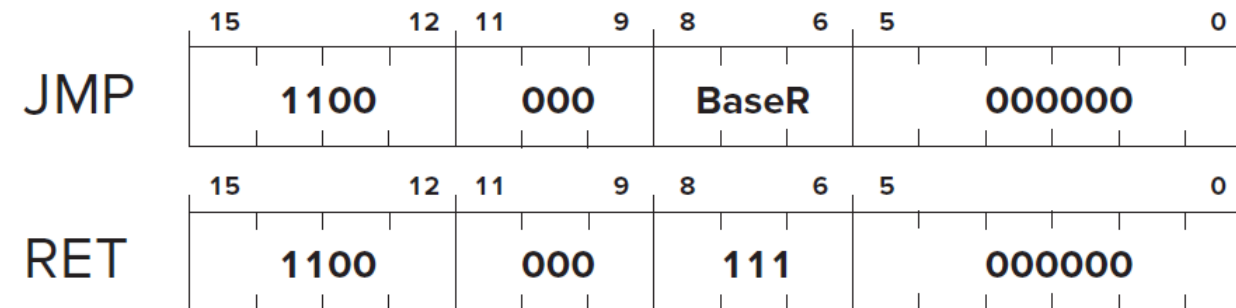
Return from Subroutine

Assembler Formats

JMP BaseR

RET

Encoding



Operation

PC = BaseR;

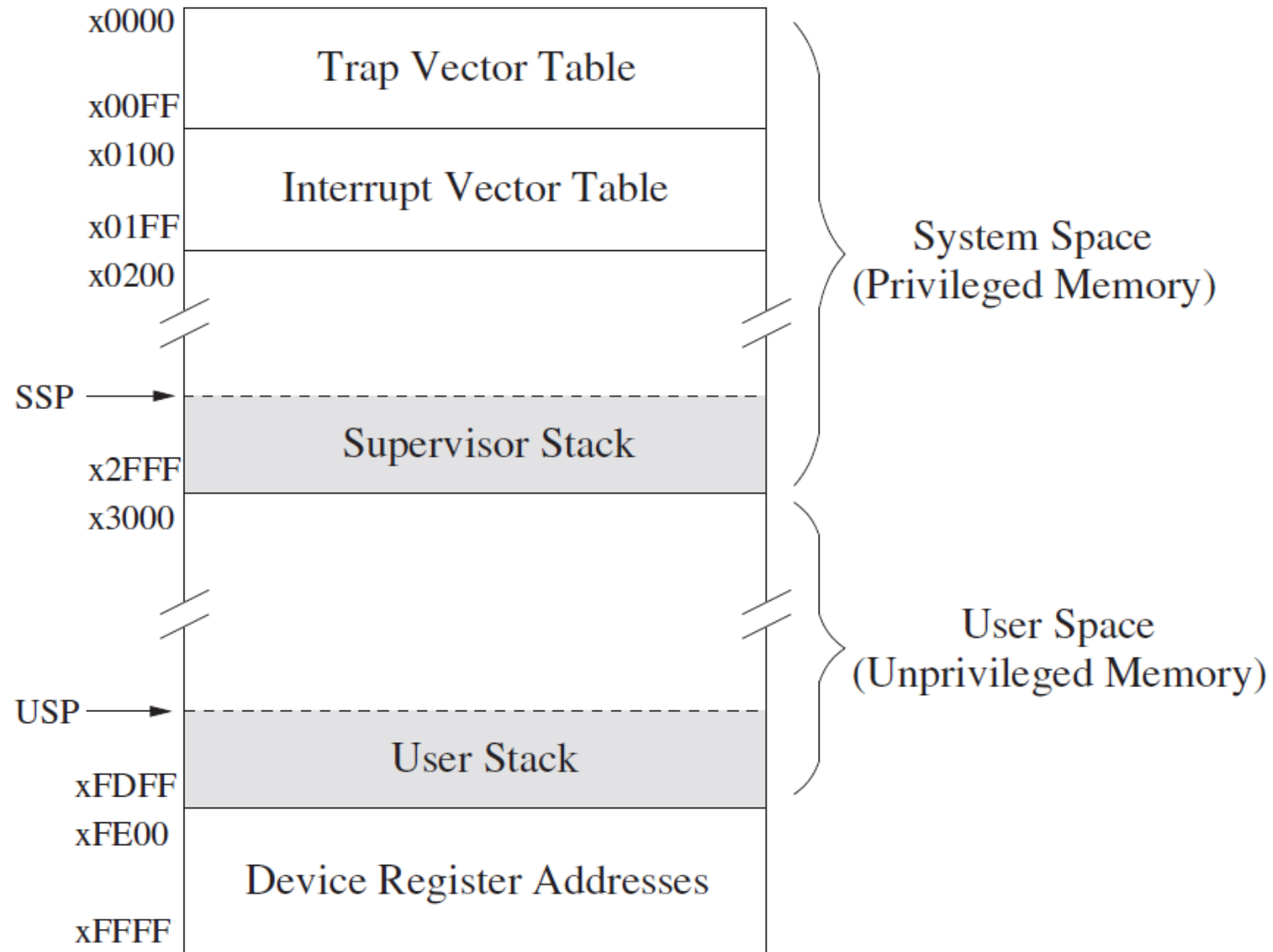


Figure A.1 Memory map of the LC-3

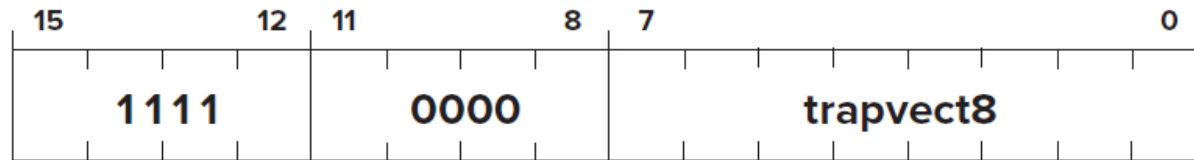
TRAP

System Call

Assembler Format

TRAP trapvect8

Encoding



Operation

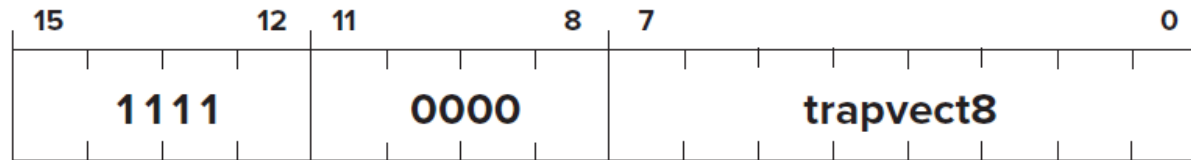
```
TEMP=PSR;  
if (PSR[15] == 1)  
    Saved_USP=R6 and R6=Saved_SSP;  
    PSR[15]=0;  
Push TEMP, PC† on the system stack  
PC=mem[ZEXT(trapvect8)];
```

TRAP

Assembler Format

TRAP trapvector8

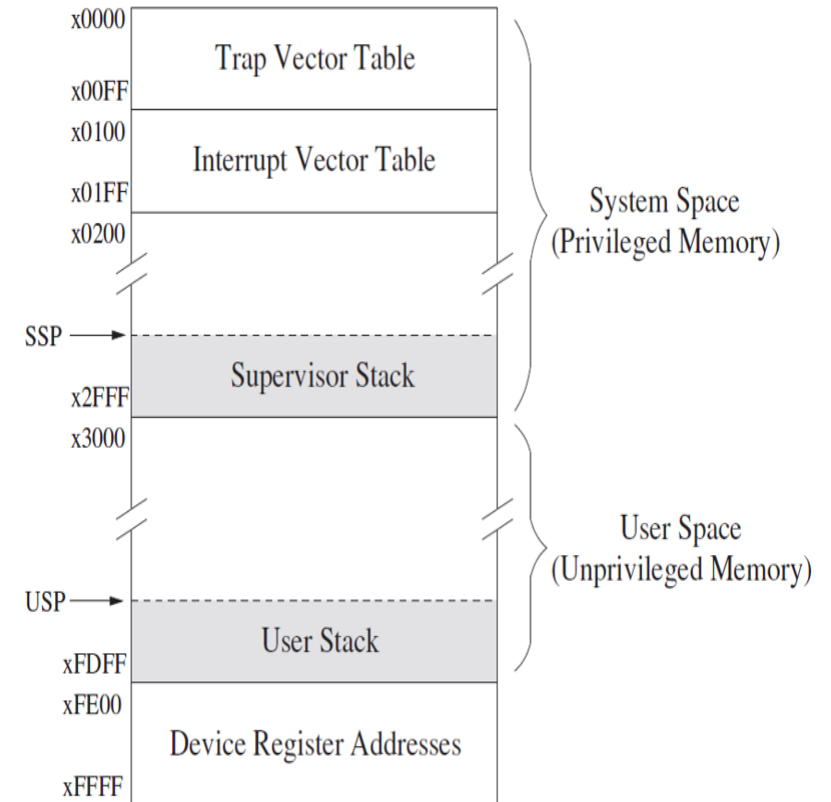
Encoding



Operation

```
TEMP=PSR;  
if (PSR[15] == 1)  
    Saved_USP=R6 and R6=Saved_SSP;  
    PSR[15]=0;  
Push TEMP,PC† on the system stack  
PC=mem[ZEXT(trapvect8)];
```

System Call



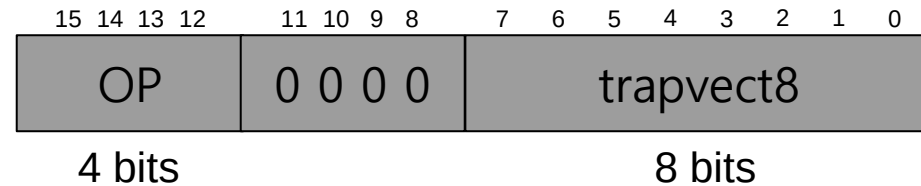
TRAP Instruction

- TRAP invokes an OS service call

LC-3 assembly

```
TRAP 0x23;
```

Machine Code



□ OP = 1111

□ trapvect8 = service call

- 0x23 = Input a character from the keyboard
- 0x21 = Output a character to the monitor
- 0x25 = Halt the program

Table A.3 **Trap Service Routines**

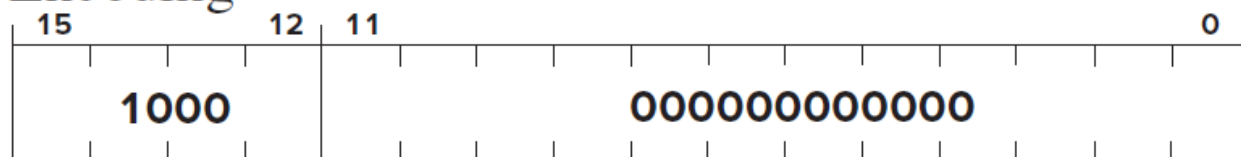
Trap Vector	Assembler Name	Description
x20	GETC	Read a single character from the keyboard. The character is not echoed onto the console. Its ASCII code is copied into R0. The high eight bits of R0 are cleared.
x21	OUT	Write a character in R0[7:0] to the console display.
x22	PUTS	Write a string of ASCII characters to the console display. The characters are contained in consecutive memory locations, one character per memory location, starting with the address specified in R0. Writing terminates with the occurrence of x0000 in a memory location.
x23	IN	Print a prompt on the screen and read a single character from the keyboard. The character is echoed onto the console monitor, and its ASCII code is copied into R0. The high eight bits of R0 are cleared.
x24	PUTSP	Write a string of ASCII characters to the console. The characters are contained in consecutive memory locations, two characters per memory location, starting with the address specified in R0. The ASCII code contained in bits [7:0] of a memory location is written to the console first. Then the ASCII code contained in bits [15:8] of that memory location is written to the console. (A character string consisting of an odd number of characters to be written will have x00 in bits [15:8] of the memory location containing the last character to be written.) Writing terminates with the occurrence of x0000 in a memory location.
x25	HALT	Halt execution and print a message on the console.

RTI

Return from Trap or Interrupt

Assembler Format

RTI Encoding



Operation

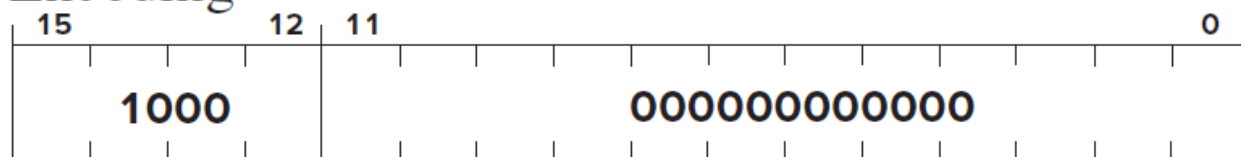
```
if (PSR[15] == 1)
    Initiate a privilege mode exception;
else
    PC=mem[R6]; R6 is the SSP, PC is restored
    R6=R6+1;
    TEMP=mem[R6];
    R6=R6+1; system stack completes POP before saved PSR is restored
    PSR=TEMP; PSR is restored
    if (PSR[15] == 1)
        Saved_SSP=R6 and R6=Saved_USP;
```

RTI

Return from Trap or Interrupt

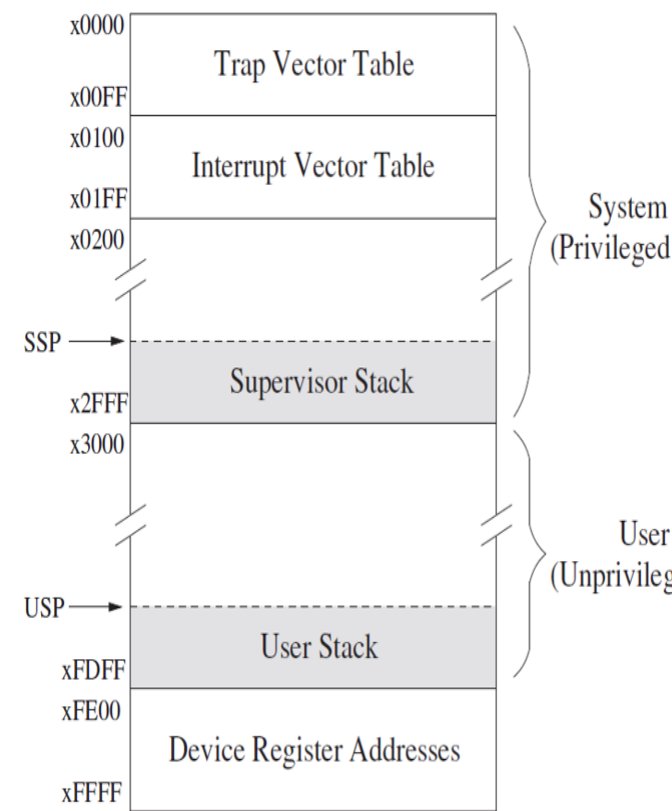
Assembler Format

RTI Encoding



Operation

```
if (PSR[15] == 1)
    Initiate a privilege mode exception;
else
    PC=mem[R6]; R6 is the SSP, PC is restored
    R6=R6+1;
    TEMP=mem[R6];
    R6=R6+1; system stack completes POP before saved PSR is restored
    PSR=TEMP; PSR is restored
    if (PSR[15] == 1)
        Saved_SSP=R6 and R6=Saved_USP;
```



ADD

Addition

Assembler Formats

```
ADD DR, SR1, SR2
ADD DR, SR1, imm5
```

Encodings

15	12	11	9	8	6	5	4	3	2	0
0001	DR	SR1	0	00	SR2					

15	12	11	9	8	6	5	4			0
0001	DR	SR1	1	imm5						

Operation

```
if (bit[5] == 0)
    DR=SR1+SR2;
else
    DR=SR1+SEXT(imm5);
setcc();
```

AND

Bit-wise Logical AND

Assembler Formats

```
AND DR, SR1, SR2
AND DR, SR1, imm5
```

Encodings

15	12	11	9	8	6	5	4	3	2	0
0101	DR	SR1	0	00	SR2					

15	12	11	9	8	6	5	4			0
0101	DR	SR1	1	imm5						

Operation

```
if (bit[5] == 0)
    DR=SR1 AND SR2;
else
    DR=SR1 AND SEXT(imm5);
setcc();
```

NOT

Bit-Wise Complement

Assembler Format

```
NOT DR, SR
```

Encoding

15	12	11	9	8	6	5	4	3	2	0
1001	DR	SR	1	1111						

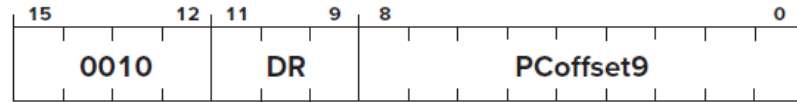
LD

Load

Assembler Format

LD DR, LABEL

Encoding



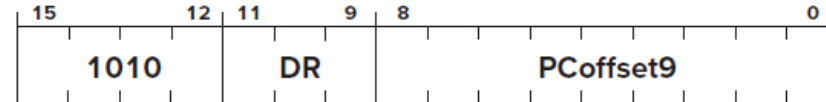
LDI

Load Indirect

Assembler Format

LDI DR, LABEL

Encoding



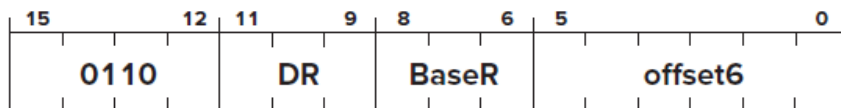
LDR

Load Base+offset

Assembler Format

LDR DR, BaseR, offset6

Encoding



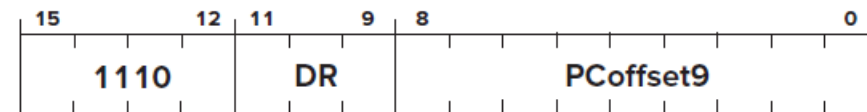
LEA

Load Effective Address

Assembler Format

LEA DR, LABEL

Encoding

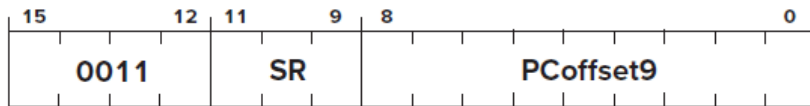


ST

Assembler Format

ST SR, LABEL

Encoding



Store

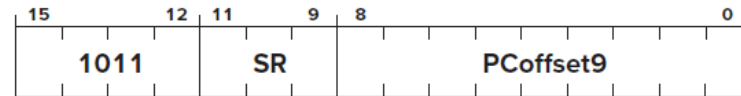
STI

Store Indirect

Assembler Format

STI SR, LABEL

Encoding



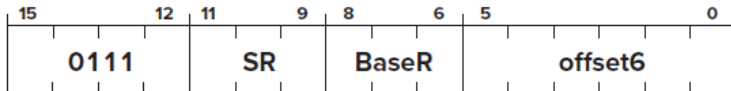
STR

Store Base+offset

Assembler Format

STR SR, BaseR, offset6

Encoding



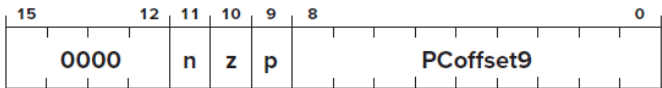
BR

Conditional Branch

Assembler Formats

BRn LABEL BRzp LABEL
BRz LABEL BRnp LABEL
BRp LABEL BRnz LABEL
BR[†] LABEL BRnzp LABEL

Encoding



JSR

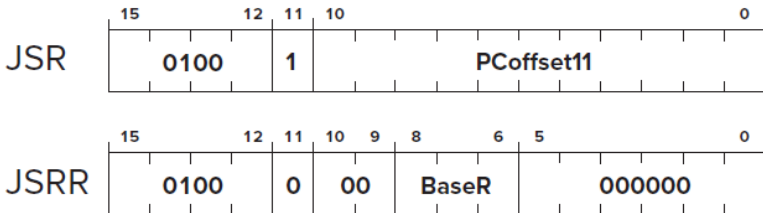
JSRR

Jump to Subroutine

Assembler Formats

JSR LABEL
JSRR BaseR

Encoding



JMP

RET

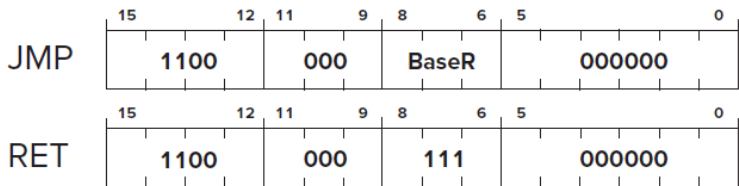
Jump

Return from Subroutine

Assembler Formats

JMP BaseR
RET

Encoding

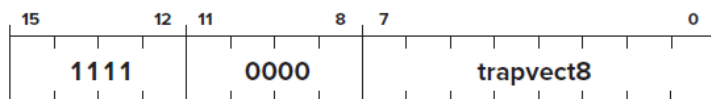


TRAP

Assembler Format

```
TRAP trapvector8
```

Encoding



Operation

```
TEMP=PSR;
if (PSR[15] == 1)
    Saved_USP=R6 and R6=Saved_SSP;
    PSR[15]=0;
Push TEMP,PC† on the system stack
PC=mem[ZEXT(trapvect8)];
```

System Call

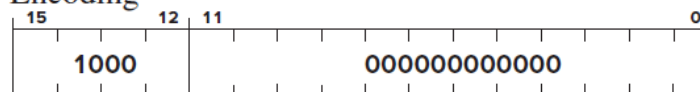
RTI

Return from Trap or Interrupt

Assembler Format

RTI

Encoding



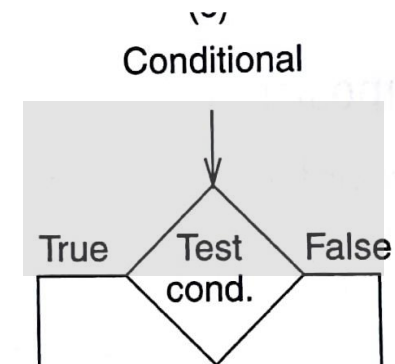
Operation

```
if (PSR[15] == 1)
    Initiate a privilege mode exception;
else
    PC=mem[R6]; R6 is the SSP, PC is restored
    R6=R6+1;
    TEMP=mem[R6];
    R6=R6+1; system stack completes POP before saved PSR is restored
    PSR=TEMP; PSR is restored
    if (PSR[15] == 1)
        Saved_SSP=R6 and R6=Saved_USP;
```


Assembly Programming

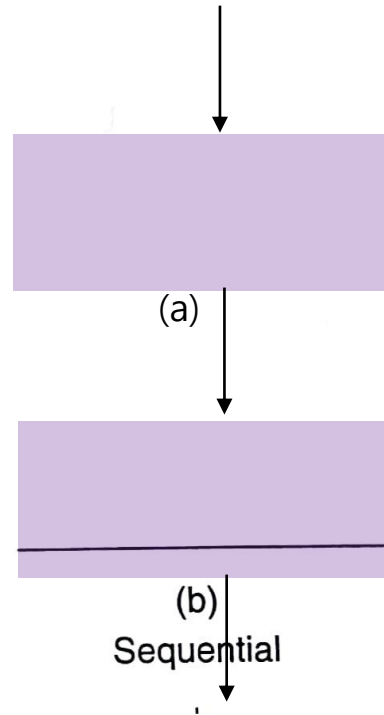
Programming Constructs

- Programming requires **dividing a task**, i.e., a unit of work into **smaller units of work**
- The goal is to replace the units of work with **programming constructs** that represent that part of the task
- There are **three basic programming constructs**
 - Sequential construct
 - Conditional construct
 - Iterative construct



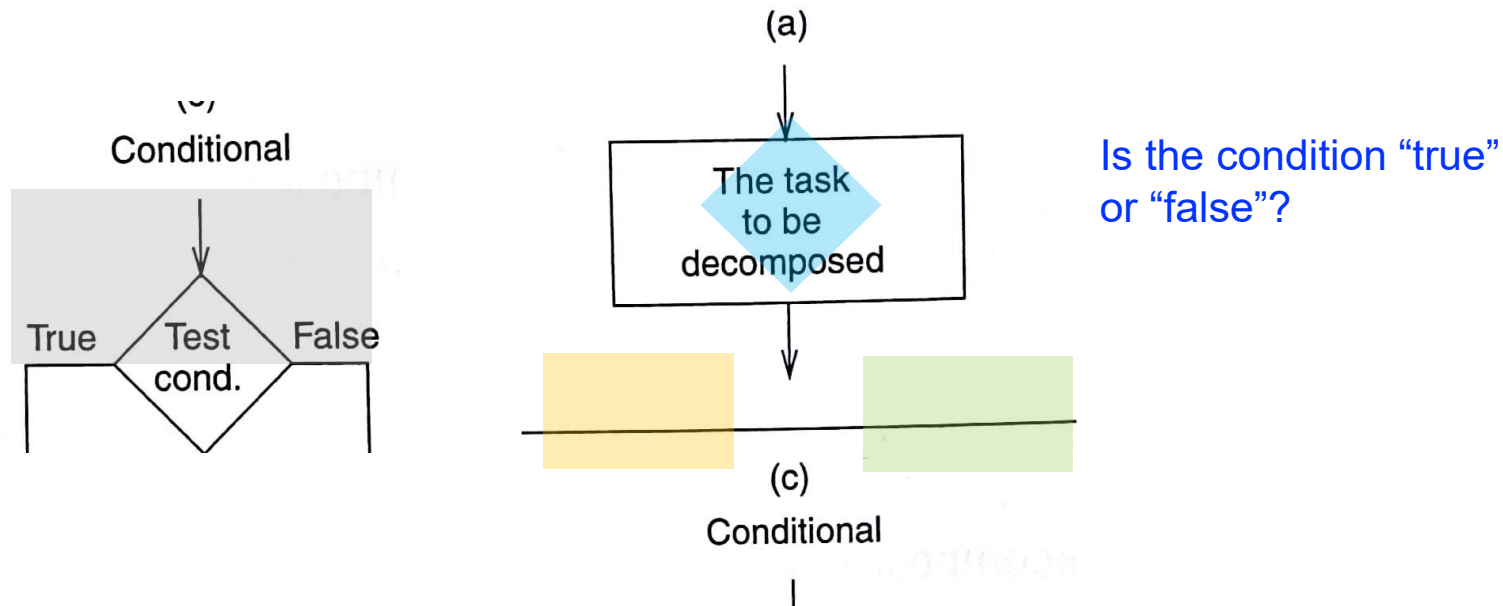
Sequential Construct

- The sequential construct is used if the designated task can be broken down into two subtasks, one following the other



Conditional Construct

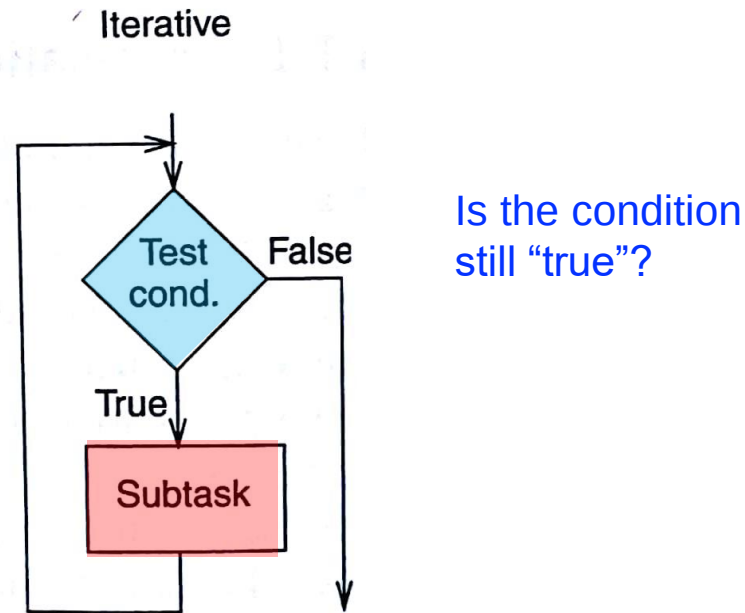
- The conditional construct is used if the designated task consists of **doing one of two subtasks**, **but not both**



- Either subtask may be **"do nothing"**
- After the correct subtask is **completed**, the program **moves onward**
- E.g., if-else statement, switch-case statement

Iterative Construct

- The iterative construct is used if the designated task consists of **doing a subtask a number of times**, but only **as long as some condition is true**



- E.g., for loop, while loop, do-while loop

Constructs in an Example Program

- Let us see how to use the programming constructs in an example program
- The example program counts the number of occurrences of a character in a text file
- It uses sequential, conditional, and iterative constructs
- We will see how to write conditional and iterative constructs with conditional branches

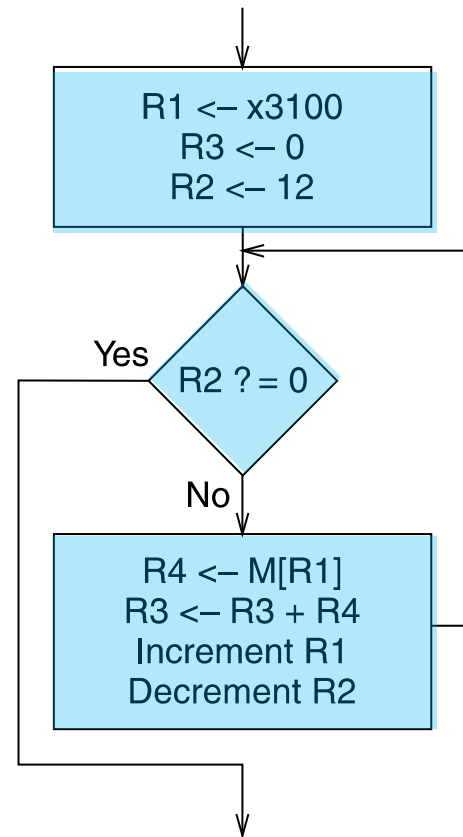
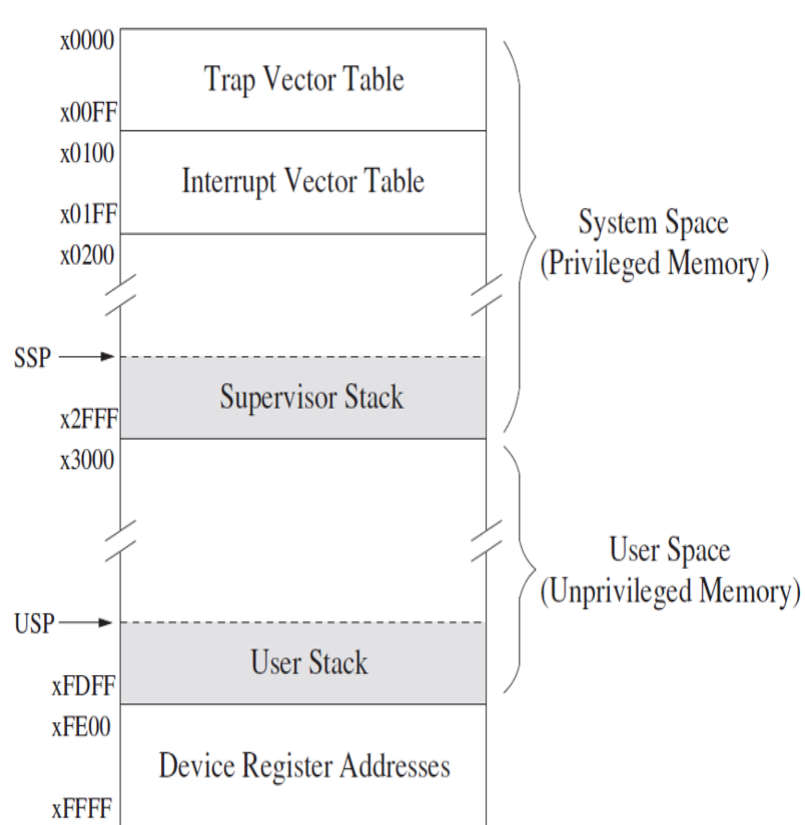
First LC-3 Program: Use of Conditional Branches for Looping

Readings

- *Introduction to Computing Systems: From Bits and Gates to C and Beyond* by Patt & Patel
 - Chapter 5, 6
 - Appendix A – LC3 ISA

An Algorithm for Adding Integers

- We want to write a program that adds 12 integers
 - They are stored in addresses 0x3100 to 0x310B
 - Let us take a look at the flowchart of the algorithm



R1: initial address of integers

R3: final result of addition

R2: number of integers
left to be added

Check if R2 becomes 0
(done with all integers?)

Load integer in R4

Accumulate integer value in R3

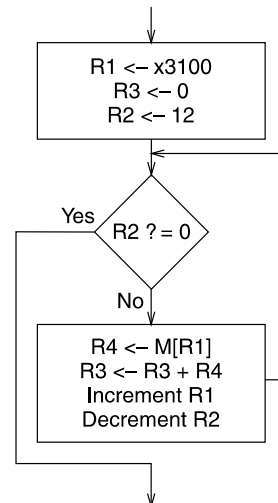
Increment address R1

Decrement R2

A Program for Adding Integers in LC-3

- We use conditional branch instructions to create a loop

Address	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
x3000	LEA	1	1	0	0	0	1	0	1	1	1	1	1	1	1	1	$R1 = PC^+ + 0x00FF = 3100$ // load address
x3001	AND	1	0	1	0	1	1	0	1	1	1	0	0	0	0	0	$R3 = 0$ // reset register
x3002	AND	1	0	1	0	1	0	0	1	0	1	0	0	0	0	0	$R2 = 0$ // reset register
x3003	ADD	0	0	1	0	1	0	0	1	0	1	0	1	1	0	0	$R2 = R2 + 12$ // initialize counter
x3004	BR	0	0	0	0	z	0	5	0	0	0	0	0	0	1	0	$BRz(PC^+ + 5) = BRz 0x300A$ // check condition
x3005	LDR	1	1	0	1	0	0	0	0	1	0	0	0	0	0	0	$R4 = M[R1 + 0]$ // load value
x3006	ADD	0	0	1	0	1	1	0	1	1	0	0	0	1	0	0	$R3 = R3 + R4$ // accumulate
x3007	ADD	0	0	1	0	0	1	0	0	1	1	0	0	0	0	1	$R1 = R1 + 1$ // increment address
x3008	ADD	0	0	1	0	1	0	0	1	0	1	1	1	1	1	1	$R2 = R2 - 1$ // decrement counter
x3009	BR	0	0	0	n	z	p	6	1	1	1	1	1	0	1	0	$BRnzp(PC^+ - 6) = BRnzp 0x3004$ // jump

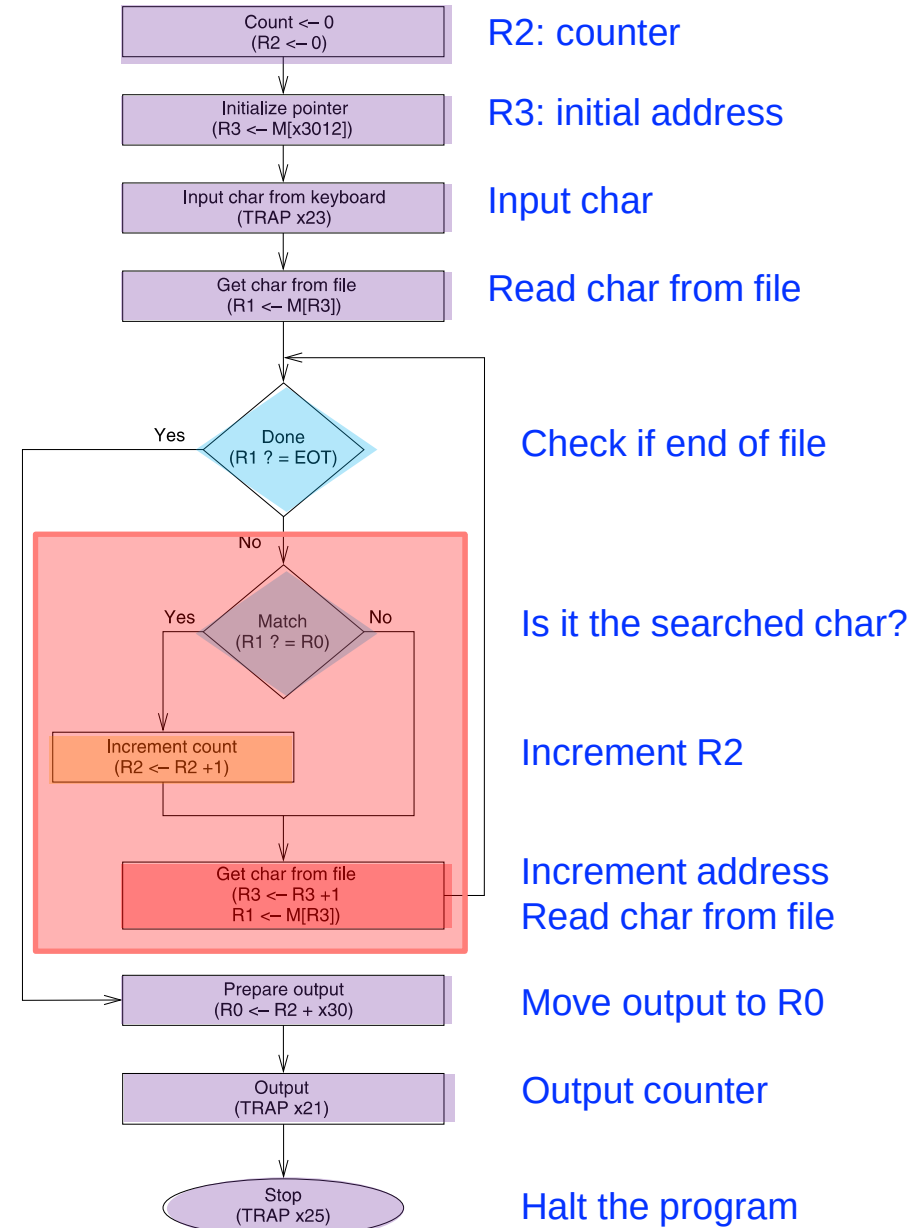


Bit 5 to differentiate both ADD instructions

[†] This is the incremented PC

Counting Occurrences of a Character

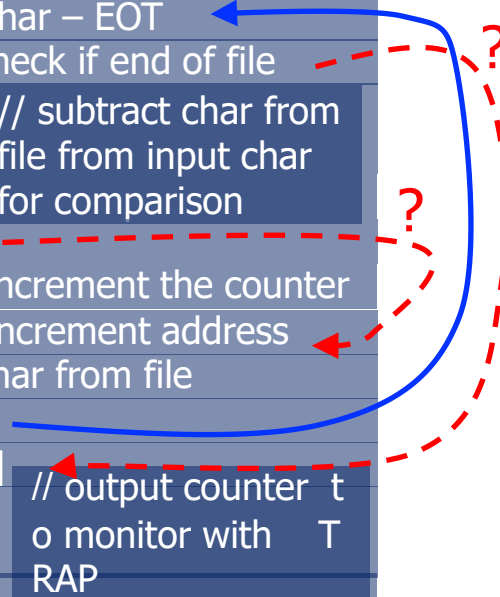
- We want to write a program that counts the occurrences of a character in a file
 - Character from the keyboard (TRAP instr.)
 - The file finishes with the character EOT (End Of Text)
 - That is called a sentinel
 - In this example, EOT = 4
 - Result to the monitor (TRAP instr.)



Counting Occurrences of a Char in LC-3

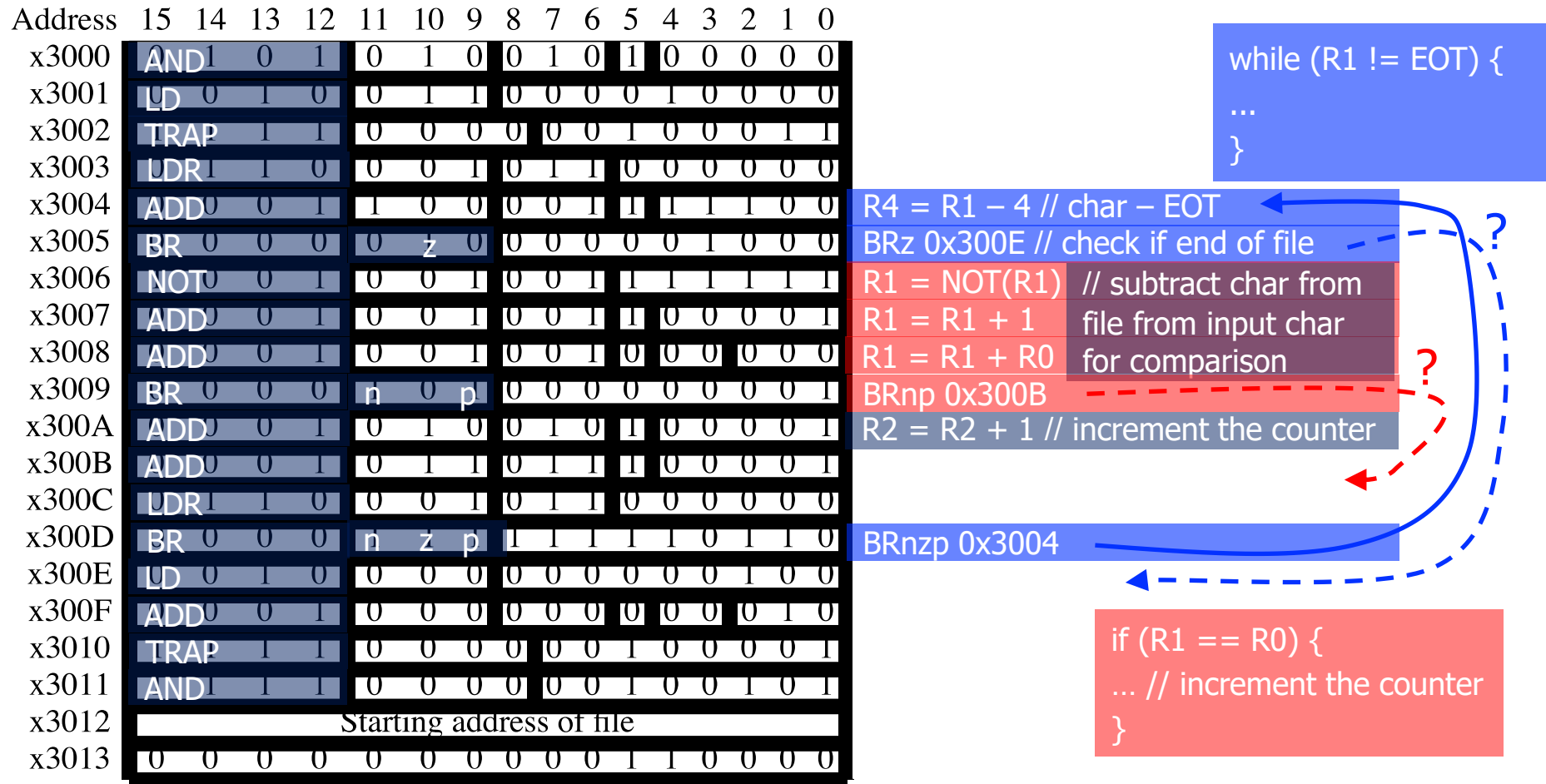
- We use **conditional branch instructions** to create a **loops** and **if statements**

Address	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
x3000	AND	1	0	1	0	1	0	0	1	0	1	0	0	0	0	0	R2 = 0 // initialize counter
x3001	LD	0	1	0	0	1	1	0	0	0	0	1	0	0	0	0	R3 = M[0x3012] // initial address
x3002	TRAP	1	1	0	0	0	0	0	0	1	0	0	0	0	1	1	TRAP 0x23 // input char to R0
x3003	LDR	1	1	0	0	0	1	0	1	1	0	0	0	0	0	0	R1 = M[R3] // char from file
x3004	ADD	0	0	1	1	0	0	0	0	1	1	1	1	1	0	0	R4 = R1 - 4 // char - EOT
x3005	BR	0	0	0	0	z	0	0	0	0	0	0	1	0	0	0	BRz 0x300E // check if end of file
x3006	NOT	0	0	1	0	0	1	0	0	1	1	1	1	1	1	1	R1 = NOT(R1) // subtract char from
x3007	ADD	0	0	1	0	0	1	0	0	1	1	0	0	0	0	1	file from input char
x3008	ADD	0	0	1	0	0	1	0	0	1	0	0	0	0	0	0	R1 = R1 + R0 for comparison
x3009	BR	0	0	0	n	0	p	0	0	0	0	0	0	0	0	1	BRnp 0x300B
x300A	ADD	0	0	1	0	1	0	0	1	0	1	0	0	0	0	1	R2 = R2 + 1 // increment the counter
x300B	ADD	0	0	1	0	1	1	0	1	1	1	0	0	0	0	1	R3 = R3 + 1 // increment address
x300C	LDR	1	1	0	0	0	1	0	1	1	0	0	0	0	0	0	R1 = M[R3] // char from file
x300D	BR	0	0	0	n	z	p	1	1	1	1	1	0	1	1	0	BRnzp 0x3004
x300E	LD	0	1	0	0	0	0	0	0	0	0	0	0	1	0	0	R0 = M[0x3013] // output counter t
x300F	ADD	0	0	1	0	0	0	0	0	0	0	0	0	0	1	0	o monitor with T
x3010	TRAP	1	1	0	0	0	0	0	0	1	0	0	0	0	0	1	TRAP 0x21
x3011	AND	1	1	1	0	0	0	0	0	0	1	0	0	1	0	1	TRAP 0x25
x3012	Starting address of file																
x3013	ASCII TEMPLATE																



Programming Constructs in LC-3

- Let us do some reverse engineering to identify **conditional constructs** and **iterative constructs**



[illegible]